

soc 111 - su 2024 - 01b

GR

2024-08-08

Note, I generated these notes using RMarkdown. It lets you easily combine text, code, and output. It generates PDFs, HTML, or DOCX. You do not need to use it for your homework or term paper, but if you're interested in trying it, please see <https://rmarkdown.rstudio.com/lesson-1.html>.

Load libraries

```
library(igraph)
```

```
##
## Attaching package: 'igraph'

## The following objects are masked from 'package:stats':
##
##   decompose, spectrum

## The following object is masked from 'package:base':
##
##   union
```

```
library(igraphdata)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x lubridate::%--%()      masks igraph::%--%()
## x dplyr::as_data_frame() masks tibble::as_data_frame(), igraph::as_data_frame()
## x purrr::compose()       masks igraph::compose()
## x tidyr::crossing()       masks igraph::crossing()
## x dplyr::filter()         masks stats::filter()
## x dplyr::lag()            masks stats::lag()
## x purrr::simplify()       masks igraph::simplify()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

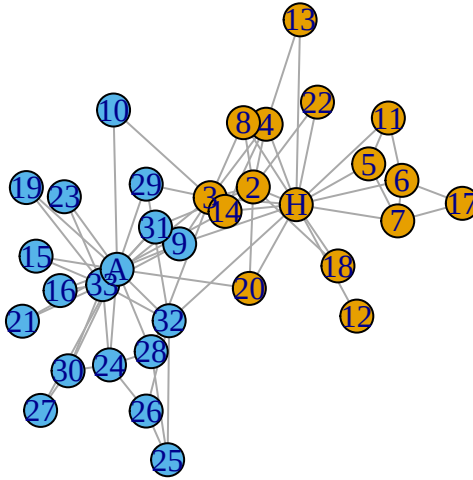
Check out the list of sample graphs in `igraphdata`.

```
data(package="igraphdata")$results
```

```
##      Package      LibPath
## [1,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [2,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [3,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [4,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [5,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [6,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [7,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [8,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [9,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [10,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
## [11,] "igraphdata" "C:/Users/gabri/AppData/Local/R/win-library/4.3"
##      Item      Title
## [1,] "Koenigsberg" "Bridges of Koenigsberg from Euler's times"
## [2,] "UKfaculty"   "Friendship network of a UK university faculty"
## [3,] "USairports"  "US airport network, 2010 December"
## [4,] "enron"       "Enron Email Network"
## [5,] "foodwebs"    "A collection of food webs"
## [6,] "immuno"      "Immunoglobulin interaction network"
## [7,] "karate"      "Zachary's karate club network"
## [8,] "kite"        "Krackhardt's kite"
## [9,] "macaque"     "Visuotactile brain areas and connections"
## [10,] "rfid"       "Hospital encounter network data"
## [11,] "yeast"      "Yeast protein interaction network"
```

Load and plot Zachary's karate club.

```
data("karate")
karate.g <- upgrade_graph(karate) # update the format and assign to new object karate.g
plot.igraph(karate.g) # draw it
```



Nodes (or vertices)

```
vcount(karate.g) # how many nodes or vertices (people)
```

```
## [1] 34
```

```
V(karate.g)$name
```

```
## [1] "Mr Hi" "Actor 2" "Actor 3" "Actor 4" "Actor 5" "Actor 6"
## [7] "Actor 7" "Actor 8" "Actor 9" "Actor 10" "Actor 11" "Actor 12"
## [13] "Actor 13" "Actor 14" "Actor 15" "Actor 16" "Actor 17" "Actor 18"
## [19] "Actor 19" "Actor 20" "Actor 21" "Actor 22" "Actor 23" "Actor 24"
## [25] "Actor 25" "Actor 26" "Actor 27" "Actor 28" "Actor 29" "Actor 30"
## [31] "Actor 31" "Actor 32" "Actor 33" "John A"
```

Edges (or ties)

Every edge is also a dyad. Often when we talk about dyads we mean disconnected dyads.

```
ecount(karate.g) # how many edges (relationships)
```

```
## [1] 78
```

```
E(karate.g) # all the edges in the network
```

```
## + 78/78 edges from 4b458a1 (vertex names):  
## [1] Mr Hi --Actor 2 Mr Hi --Actor 3 Mr Hi --Actor 4 Mr Hi --Actor 5  
## [5] Mr Hi --Actor 6 Mr Hi --Actor 7 Mr Hi --Actor 8 Mr Hi --Actor 9  
## [9] Mr Hi --Actor 11 Mr Hi --Actor 12 Mr Hi --Actor 13 Mr Hi --Actor 14  
## [13] Mr Hi --Actor 18 Mr Hi --Actor 20 Mr Hi --Actor 22 Mr Hi --Actor 32  
## [17] Actor 2--Actor 3 Actor 2--Actor 4 Actor 2--Actor 8 Actor 2--Actor 14  
## [21] Actor 2--Actor 18 Actor 2--Actor 20 Actor 2--Actor 22 Actor 2--Actor 31  
## [25] Actor 3--Actor 4 Actor 3--Actor 8 Actor 3--Actor 9 Actor 3--Actor 10  
## [29] Actor 3--Actor 14 Actor 3--Actor 28 Actor 3--Actor 29 Actor 3--Actor 33  
## [33] Actor 4--Actor 8 Actor 4--Actor 13 Actor 4--Actor 14 Actor 5--Actor 7  
## [37] Actor 5--Actor 11 Actor 6--Actor 7 Actor 6--Actor 11 Actor 6--Actor 17  
## + ... omitted several edges
```

```
# in class the code i did to print one edge per line didn't work  
# this code could be adapted to do it, but probably not worth it  
# https://stackoverflow.com/questions/45403227/print-text-vector-line-by-line-in-r
```

Triads

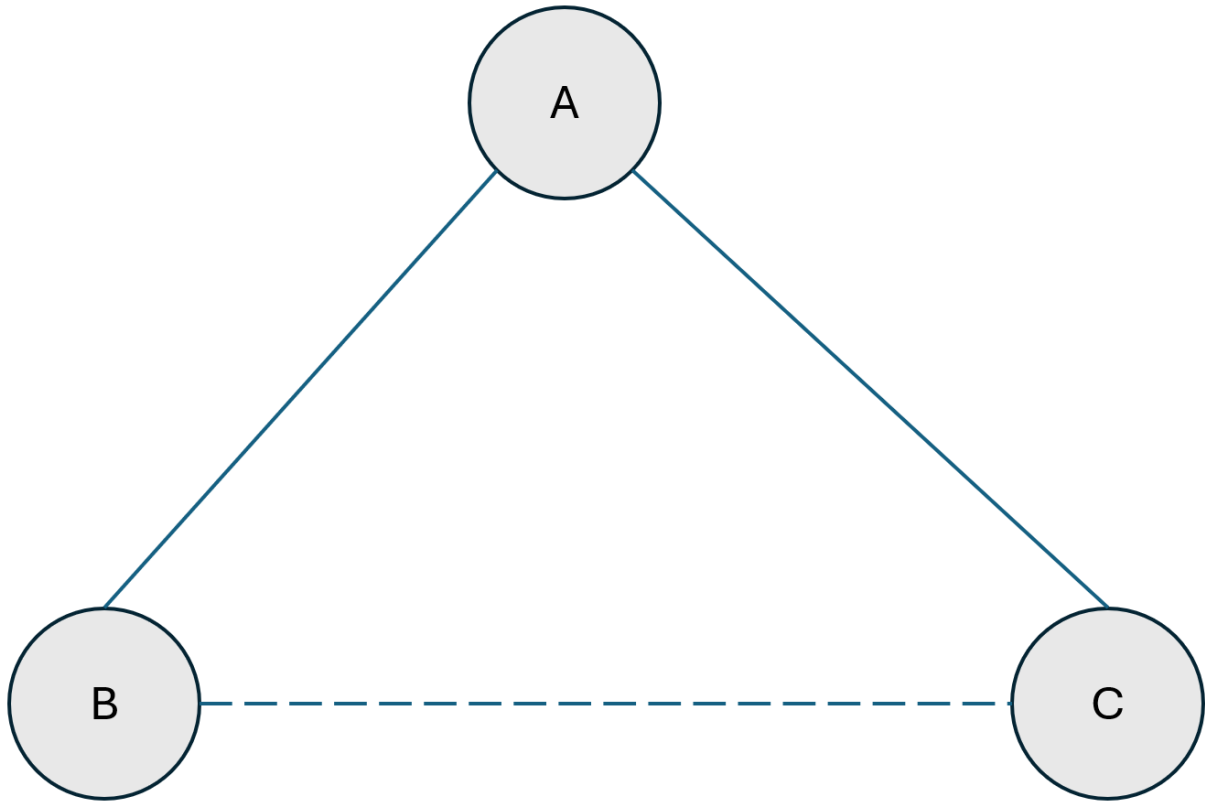
```
triangles(karate.g) %>% matrix(nrow = 3) %>% dim()
```

```
## [1] 3 45
```

```
triangles(karate.g) %>% matrix(nrow = 3) %>% ncol()
```

```
## [1] 45
```

Triadic closure



Clustering Coefficient / Transitivity

```
transitivity(karate.g) # if AB and BC, how likely is AC?
```

```
## [1] 0.2556818
```

Density

$$d(G)^u = \frac{m}{\frac{n(n-1)}{2}} = m * \frac{2}{n(n-1)} = \frac{2m}{n(n-1)}$$

```
edge_density(karate.g, loops = F)
```

```
## [1] 0.1390374
```

```
k.m <- ecount(karate.g)
k.v <- vcount(karate.g)
k.m / ((k.v*(k.v-1))/2)
```

```
## [1] 0.1390374
```

```
mean(degree(karate.g)) / (vcount(karate.g)-1)
```

```
## [1] 0.1390374
```

```
karate.mat <- as_adjacency_matrix(karate.g, type = "lower")
karate.mat
```

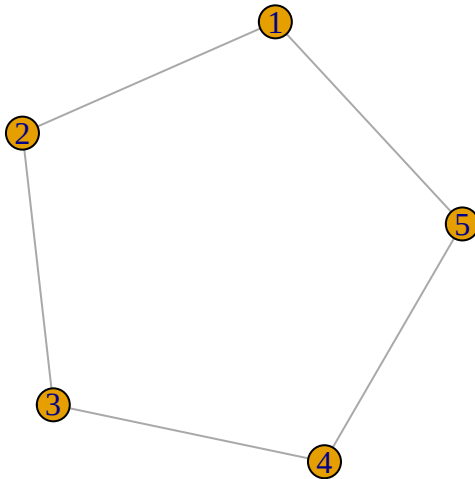
```
## 34 x 34 sparse Matrix of class "dgCMatrix"
```

```
## [[ suppressing 34 column names 'Mr Hi', 'Actor 2', 'Actor 3' ... ]]
```

```
##
## Mr Hi      . . . . .
## Actor 2    1 . . . . .
## Actor 3    1 1 . . . . .
## Actor 4    1 1 1 . . . . .
## Actor 5    1 . . . . .
## Actor 6    1 . . . . .
## Actor 7    1 . . . 1 1 . . . . .
## Actor 8    1 1 1 1 . . . . .
## Actor 9    1 . 1 . . . . .
## Actor 10   . . 1 . . . . .
## Actor 11   1 . . . 1 1 . . . . .
## Actor 12   1 . . . . .
## Actor 13   1 . . 1 . . . . .
## Actor 14   1 1 1 1 . . . . .
## Actor 15   . . . . .
## Actor 16   . . . . .
## Actor 17   . . . . . 1 1 . . . . .
## Actor 18   1 1 . . . . .
## Actor 19   . . . . .
## Actor 20   1 1 . . . . .
## Actor 21   . . . . .
## Actor 22   1 1 . . . . .
## Actor 23   . . . . .
## Actor 24   . . . . .
## Actor 25   . . . . .
## Actor 26   . . . . .      1 1 . . . . .
## Actor 27   . . . . .      1 1 . . . . .
## Actor 28   . . 1 . . . . .      1 1 . . . . .
## Actor 29   . . 1 . . . . .      1 . . 1 . . . . .
## Actor 30   . . . . .      1 . . 1 . . . . .
## Actor 31   . 1 . . . . . 1 . . . . .      1 1 . . 1 . . . . .
## Actor 32   1 . . . . .      1 . . . . .      1 1 . . 1 . . . . .
## Actor 33   . . 1 . . . . . 1 . . . . . 1 1 . . 1 . 1 1 . . . . 1 1 1 . .
## John A     . . . . . 1 1 . . . 1 1 1 . . 1 1 1 . 1 1 . . 1 1 1 1 1 1 1 .
```

A smaller example.

```
ring.g <- make_ring(5)
plot.igraph(ring.g)
```



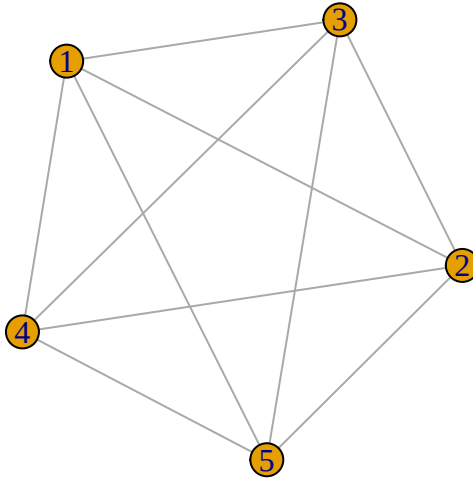
```
as_adjacency_matrix(ring.g, type = "lower")
```

```
## 5 x 5 sparse Matrix of class "dgCMatrix"
##
## [1,] . . . . .
## [2,] 1 . . . .
## [3,] . 1 . . .
## [4,] . . 1 . .
## [5,] 1 . . 1 .
```

```
edge_density(ring.g)
```

```
## [1] 0.5
```

```
ego <- seq(1:5)
alter <- ego
clique.g <- tidyr::crossing(ego,alter) %>% filter(ego!=alter) %>% graph_from_data_frame(directed = F) %>%
plot.igraph(clique.g)
```



```
as_adjacency_matrix(clique.g, type = "lower")
```

```
## 5 x 5 sparse Matrix of class "dgCMatrix"
##   1 2 3 4 5
## 1 . . . . .
## 2 1 . . . .
## 3 1 1 . . .
## 4 1 1 1 . .
## 5 1 1 1 1 .
```

```
edge_density(clique.g)
```

```
## [1] 1
```

Getting data on igraph

There are several ways to get a network into R/igraph. Here are the most important for this class:

- load from a package like `igraphdata`
 - first use `library()` to load the package
 - then use `data()` to load the specific data frame
 - optional: use `upgrade_graph()` to ensure the format is in the current `igraph` format

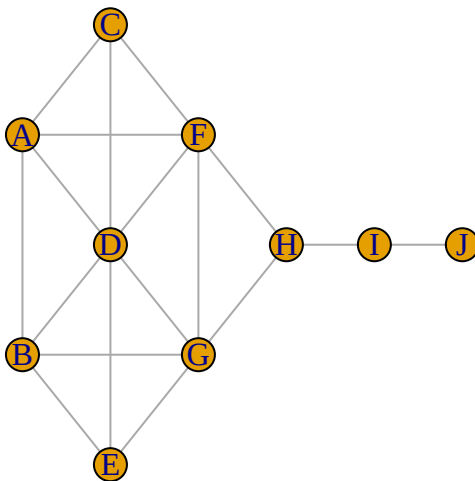
- load from a saved R data object
 - `read_rds()` is for files named `foo.rds` and requires you to assign the data on disk to an object
 - `load()` is for files named `foo.Rdata` and can have multiple objects, each already named
- load a data frame (spreadsheet) from disk and then treat as an edge list
 - You will need to use this for your term paper.
- generate an algorithmic graph
 - `igraph` has various functions starting with `make_...` or `sample_...` to create new graphs
 - eg, `make_ring(n)` for a ring and `sample_gnp(n,p)` for an Erdos-Renyi graph
 - `make_...` functions are deterministic and so always generate identical graphs whereas `sample_...` functions are stochastic and so each time will be similar but a bit different.

Here are some examples.

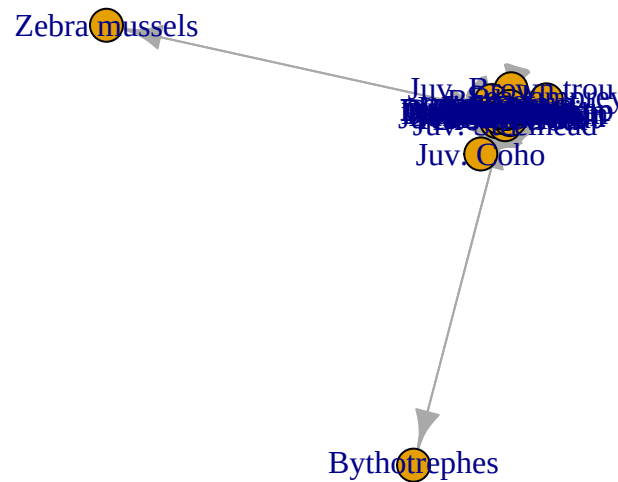
Load from a package

```
library(igraphdata)

data("kite")
kite.g <- upgrade_graph(kite) %>% igraph::simplify()
plot.igraph(kite.g)
```



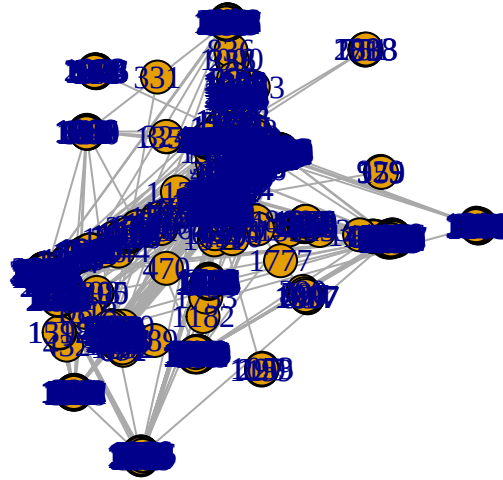
```
data("foodwebs")
# note that foodwebs is not just one graph but a list of graphs so we need to tell it which one we want
foodwebs.g <- upgrade_graph(foodwebs[["Michigan"]]) %>% igraph::simplify()
plot.igraph(foodwebs.g)
```



Load from a saved R object

```
load("dnc_igraph.Rdata")
plot.igraph(dnc)
```

```
## This graph was created by an old(er) igraph version.
## Call upgrade_graph() on it to use with the current igraph version
## For now we convert it on the fly...
```



Load from a data frame and treat as an edge list

Remember, this is very likely how you will get your data into R for your term paper.

```
dl.df <- read_csv("dysonlogos_shaftdungeon.txt")
```

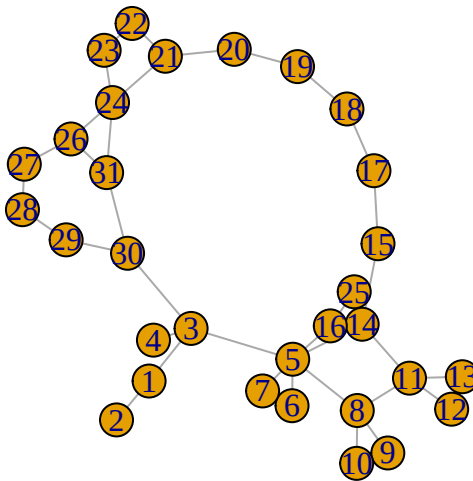
```
## Rows: 36 Columns: 3
## -- Column specification -----
## Delimiter: ","
## dbf (3): ego, alter, climb
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
dl.df[1:10,] # check out first few rows of the edge list
```

```
## # A tibble: 10 x 3
##   ego alter climb
##   <dbl> <dbl> <dbl>
## 1     1     2     0
## 2     1     3     0
## 3     3     4     0
## 4     3    30     0
## 5     3     5     0
```

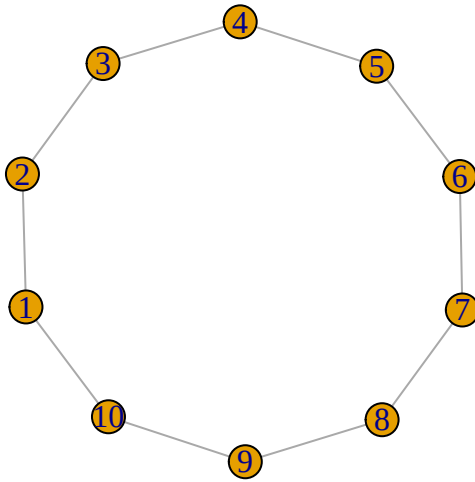
```
## 6      5      6      0
## 7      5      7      0
## 8      5      8      0
## 9      8      9      0
## 10     8     10      0
```

```
d1.g <- graph_from_data_frame(d1.df, directed = F) %>% igraph::simplify()
plot.igraph(d1.g)
```



Generate an algorithmic graph

```
ring10.g <- make_ring(10)
plot.igraph(ring10.g)
```



```
ws.g <- sample_smallworld(1,20,3,.05)  
plot.igraph(ws.g)
```

